

the PCBs. The time involved in delay and closure of relays are in seconds, still the designers used a conventional design approach of one microcontroller driving the 31 relays. This left no space for interfacing the pots (which needed ADCs), and they added another microcontroller to just do this function and complicated the design with two microcontrollers. The whole design became complicated and expensive to maintain with two software in the same product. This step could have been avoided had the designers spent just a day or two on the architecture.

The designers came to us for guidance and help in redesigning to make the whole thing simpler. After studying the requirement (major constraint was that the existing enclosure was of 15cm diameter and 10cm long cylinder and the PCBs had to be fitted inside this constrained space, our team redesigned the architecture and made the whole design simpler as shown in Fig. 2. We spent about a day in understanding the requirement and did a bottom-up new design. As you can see, the advantages of the new design are:

1. One microcontroller and simpler software
 2. Interconnects between the PCBs are reduced from 22 to 4, making the engineering simple and manufacturing easier
 3. This architecture offered an open option to add more relays for next-generation systems.
 4. Above all, being a long-life product, the design can be supported without much effort.

1. One microcontroller and simpler software

2. Interconnects between the PCBs are reduced from 22 to 4, making the engineering simple and manufacturing easier

3. This architecture offered an open option to add more relays for next-generation systems.

4. Above all, being a long-life product, the design can be supported without much effort.

What need to be factored

When designers develop product architecture, following elements need to be considered:

1. Choice of processor and the variations available with different memory sizes and the different packages available.

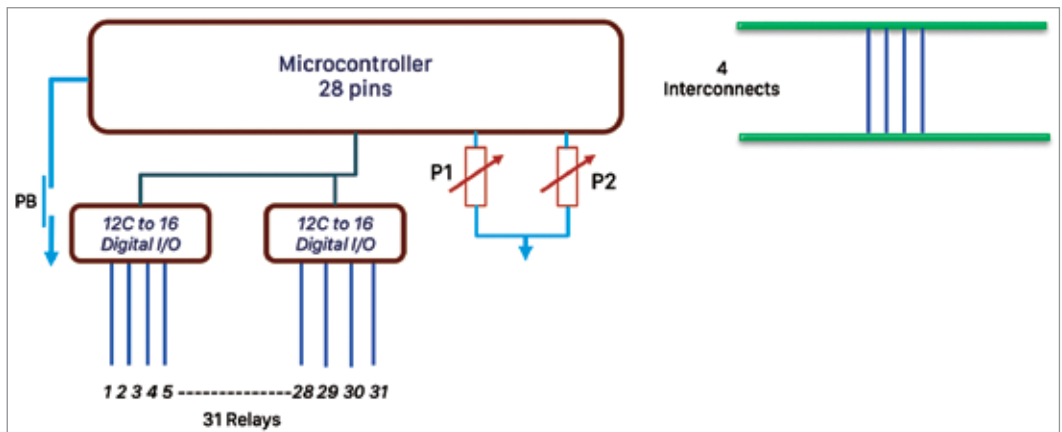


Fig. 2: The simplified design with redesigned architecture

2. Number of PCBs and partitioning of the system by functional blocks to ensure optimal design.

3. If the system is battery operated, architecture design becomes critical as battery life entirely depends on the speed of operation and quiescent current consumed by the electronics, which are proportional to the speed of the processor.

4. If the product is for regulated industries, the architecture should be either 'fault-tolerant' or 'fail-safe.' In the case of fault-tolerant system, architecture should work without stopping, which is typically needed in medical and telecom systems. This may call for use of two processors. In the case fail-safe systems, when system fails, the system should go to a safe state without harming the user.

5. Spending time in understanding the development and debugging tools available and their compliance to safety standards. Typically, most free tools or open source tools do not comply with any safety requirement and special tools have to be used.

6. Understand the processor clock speed and product performance needs to avoid selecting an under-powered speed that could compromise the system's performance.

Tips to develop the architecture

Developing an architecture for an electronic product can be a full course as it impacts both hardware and software. However, here are a few tips to use.

1. When product idea is conceived, build the product with functional blocks with each block doing a unique function.

2. Plan the interconnects between blocks with groups of signals (had the team that designed in the above example done this step they would have figured out that they had a challenge) separated as data, power, clock, etc. This gives a clear overview of the signals and brings out any missing signal.

3. If the product has to be designed with multiple PCBs (any electronic product which has a user interface always has a separate PCB for the display), identify which block will go to which PCB. Revisit the interconnects again as now you will have more signals to connect

4. Always choose a controller that has different memory options and different pinout options. This helps in customising the product with just minimal hardware and software changes.

5. When you design an IoT solution, the 'things' will be in large numbers, so the cost has to be low. However, each 'thing' will have interconnections to the gateway or cloud. The interconnection needs to be selected properly. Many designs run into cost and performance issues due to wrong choice of interface as well as wrong component.

6. If you are using vendor-supplied reference hardware or software, first understand the design well as these are meant for demonstration and their cost and memory size will never be optimised. **EFY**



S.A. Srinivasa Moorthy is director at D4X Technologies Private Limited